LEVEL 2 HACKATHON

FUNDAMENTALS OF WEB DEVELOPMENT

ONLINE QUIZ APPLICATION

QuizMaster — Learn. Answer. Improve.

Student Details	
Student Name	R. Akshaya
Register Number	25133091802522003
Student ID	423e1ca22ce4c1ca83b436d344e5d641
Department	II-B.Sc. Computer Science
College	Jawaharlal Nehru College for Women
Date of Submission	27.10.2026
GitHub Link	Add the repository link after uploading the project

Project report covering the problem statement, objectives, architecture, implementation, source code, execution, output previews, technical explanation and future enhancements.

1. Problem Statement

Many learners need a simple way to test their knowledge and receive an immediate score. Paper-based quizzes can take time to check, and learners may not have a convenient way to practise different topics. The Online Quiz Application, named QuizMaster, provides an interactive browser-based quiz where a user selects a category, answers multiple-choice questions, uses navigation controls, works within a time limit and views a final score.

2. Objectives

• To create a simple, responsive quiz website using HTML, CSS and JavaScript.

• To allow users to select General Knowledge, Web Development or Computer Science.

• To display multiple-choice questions and let users select answers.

• To provide Previous and Next navigation and a visible countdown timer.

• To randomize the question order for each quiz attempt.

• To calculate and display the final score and provide feedback.

• To allow the user to restart and attempt the quiz again.

3. Architecture

The application runs in a web browser. HTML defines the page sections and controls, CSS controls the layout and responsive appearance, and JavaScript stores the question sets, handles user actions, controls the timer and calculates the score. No server or database is required for this demonstration; the quiz runs locally in the browser.

Start / Home
Select quiz category
Start quiz and shuffle questions
Display question and options
Select answer • Previous / Next • Timer
Submit or timer reaches zero
Calculate score and show result
Try Again → Home

Technologies Used

Technology	Role
HTML5	Structures the home, quiz and result sections.
CSS3	Styles cards, buttons, options, progress bar and mobile layout.
JavaScript	Question selection, category handling, answer storage, timer, navigation and scoring.
Web browser	Runs the application locally.

4. Practical Implementation

The project is organised into three files: index.html, style.css and script.js. The home screen lets the user choose a category. When Start Quiz is clicked, the selected question set is copied and shuffled, answer slots are initialized, the quiz view is shown and a 60-second timer starts. Each answer is saved by question index, so the user can move backward and forward without losing the selected option. On submission or timeout, the application compares selected answers with the correct answers and displays the score.

4.1 Features Implemented

Feature	Purpose
Category selection	Offers General Knowledge, Web Development and Computer Science.
Dynamic questions	Displays one question and its four options at a time.
Random question order	Shuffles the questions for each new attempt.
Answer selection	Highlights the selected option and stores it.
Previous / Next	Lets users navigate between questions.
Countdown timer	Provides a 60-second quiz time limit.
Progress indicator	Shows progress through the five questions.
Result and feedback	Calculates the score out of five and shows feedback.
Restart	Returns the user to the home screen for another attempt.
Responsive design	Adjusts spacing and card layout for smaller screens.

4.2 Source Code

HTML: The HTML file creates the page header, category selector, quiz area, timer, navigation buttons and result area. It links the external stylesheet and JavaScript file.

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>QuizMaster - Online Quiz Application</title>
  <link rel="stylesheet" href="style.css">
</head>
<body>
  <main class="app">
    <header class="topbar">
      <div class="logo">Q</div>
      <div><h1>QuizMaster</h1><p>Learn. Answer. Improve.</p></div>
    </header>
    <section id="home" class="card">
      <span class="eyebrow">WELCOME TO QUIZMASTER</span>
      <h2>Test your knowledge</h2>
      <p>Choose a category and answer multiple-choice questions within the time limit.</p>
      <label for="category">Quiz category</label>
      <select id="category">
        <option value="general">General Knowledge</option>
        <option value="web">Web Development</option>
        <option value="computer">Computer Science</option>
      </select>
      <button id="startBtn">Start Quiz</button>
      <p class="small">5 questions • 60 seconds • Instant result</p>
    </section>
    <section id="quiz" class="card hidden">
      <div class="quiz-head"><span id="categoryLabel">General Knowledge</span><span id="timer">60s</span></div>
      <div class="progress"><div id="progressBar"></div></div>
      <p id="questionCount" class="small">Question 1 of 5</p>
      <h2 id="questionText"></h2>
      <div id="options" class="options"></div>
      <div class="actions"><button id="prevBtn" class="secondary">Previous</button><button id="nextBtn">Next</button></div>
    </section>
    <section id="result" class="card hidden">
      <span class="eyebrow">QUIZ COMPLETED</span><h2>Your Result</h2>
      <div class="score" id="score">0/5</div><p id="feedback"></p>
```

```
      <button id="restartBtn">Try Again</button>
    </section>
    <footer>QuizMaster • Online Quiz Application</footer>
  </main>
  <script src="script.js"></script>
</body>
</html>
```

CSS

CSS styles the application with a centered card layout, readable text, option buttons, progress bar, timer badge and responsive rules for narrow screens.

```
* { box-sizing: border-box; }
body {
  margin: 0; min-height: 100vh; font-family: Arial, sans-serif;
  background: linear-gradient(135deg, #eaf2ff, #f8faff); color: #1d2b50;
  display: flex; justify-content: center; align-items: center; padding: 24px;
}
.app { width: 100%; max-width: 720px; }
.topbar { display: flex; align-items: center; gap: 12px; margin-bottom: 20px; }
.logo { width: 46px; height: 46px; border-radius: 14px; background: #2864e8;
  color: white; display: grid; place-items: center; font-size: 24px; font-weight: bold; }
h1, h2, p { margin-top: 0; }
h1 { margin-bottom: 3px; font-size: 25px; } .topbar p { margin: 0; color: #66728c; }
.card { background: white; padding: 32px; border-radius: 20px;
  box-shadow: 0 14px 40px #233b6818; }
.eyebrow { color: #2864e8; font-size: 12px; font-weight: bold; letter-spacing: 1px; }
.card h2 { margin: 12px 0; font-size: 27px; }
.card p { color: #5f6b82; line-height: 1.6; }
label { display: block; font-weight: bold; margin: 22px 0 8px; }
select { width: 100%; padding: 13px; border: 1px solid #ccd5e5; border-radius: 10px; background: white; }
button { border: 0; border-radius: 10px; padding: 13px 20px; background: #2864e8;
  color: white; font-weight: bold; cursor: pointer; }
#startBtn, #restartBtn { width: 100%; margin-top: 18px; }
button:hover { filter: brightness(.94); } .small { font-size: 13px; }
.hidden { display: none; }
.quiz-head, .actions { display: flex; justify-content: space-between; align-items: center; gap: 12px; }
#timer { background: #fff2dc; color: #a75b00; padding: 8px 12px; border-radius: 9px; font-weight: bold; }
.progress { height: 7px; background: #e9eef8; border-radius: 8px; margin: 20px 0; overflow: hidden; }
#progressBar { height: 100%; width: 20%; background: #2864e8; transition: width .2s; }
.options { display: grid; gap: 10px; margin: 20px 0; }
.option { border: 1px solid #d6deed; border-radius: 11px; padding: 14px; cursor: pointer; }
.option.selected { border-color: #2864e8; background: #edf3ff; }
.secondary { background: #e9eef8; color: #1d2b50; }
.score { font-size: 48px; color: #2864e8; font-weight: bold; margin: 18px 0; }
footer { text-align: center; color: #78849a; font-size: 12px; margin-top: 18px; }
@media (max-width: 520px) { body { padding: 14px; } .card { padding: 22px; } .card h2 { font-size: 23px; } }
```

JavaScript

JavaScript contains the category question sets and functions to start the quiz, display questions, store answers, update the timer, calculate the score and restart the quiz.

```
const questionSets = {
  general: [
    { q: "Which is the largest planet in our solar system?", o: ["Earth", "Jupiter", "Mars", "Venus"], a: 1 },
    { q: "How many days are there in a leap year?", o: ["364", "365", "366", "367"], a: 2 },
    { q: "Which is the national animal of India?", o: ["Lion", "Elephant", "Tiger", "Peacock"], a: 2 },
    { q: "Which gas do plants absorb?", o: ["Oxygen", "Carbon dioxide", "Nitrogen", "Hydrogen"], a: 1 },
    { q: "What is the capital of Japan?", o: ["Seoul", "Beijing", "Tokyo", "Bangkok"], a: 2 }
  ],
  web: [
    { q: "What does HTML stand for?", o: ["Hyper Text Markup Language", "High Text Machine Language", "Hyper Tool Multi Language", "Home Text Mark
    { q: "Which language styles a web page?", o: ["SQL", "CSS", "C++", "Python"], a: 1 },
    { q: "Which tag creates a hyperlink?", o: ["<p>", "<img>", "<a>", "<h1>"], a: 2 },
    { q: "Which language adds interactivity to a page?", o: ["JavaScript", "HTML only", "CSS only", "XML"], a: 0 },
    { q: "Which HTML tag inserts an image?", o: ["<picturetext>", "<img>", "<src>", "<imagefile>"], a: 1 }
  ],
  computer: [
    { q: "Which part is known as the brain of a computer?", o: ["Monitor", "Keyboard", "CPU", "Mouse"], a: 2 },
    { q: "Which is an input device?", o: ["Printer", "Keyboard", "Speaker", "Monitor"], a: 1 },
    { q: "What does RAM stand for?", o: ["Read Access Memory", "Random Access Memory", "Run All Memory", "Rapid Action Module"], a: 1 },
    { q: "Which of these is an operating system?", o: ["Windows", "HTML", "Mouse", "Google"], a: 0 },
    { q: "Which number system uses 0 and 1?", o: ["Decimal", "Octal", "Binary", "Roman"], a: 2 }
  ]
};
const home = document.getElementById("home"), quiz = document.getElementById("quiz");
const result = document.getElementById("result"), category = document.getElementById("category");
const questionText = document.getElementById("questionText"), optionsBox = document.getElementById("options");
const questionCount = document.getElementById("questionCount"), timerLabel = document.getElementById("timer");
const progressBar = document.getElementById("progressBar");
let questions = [], current = 0, answers = [], seconds = 60, timerId;
document.getElementById("startBtn").addEventListener("click", startQuiz);
document.getElementById("prevBtn").addEventListener("click", () => showQuestion(current - 1));
document.getElementById("nextBtn").addEventListener("click", () => {
  if (current < questions.length - 1) showQuestion(current + 1); else finishQuiz();
});
document.getElementById("restartBtn").addEventListener("click", () => {
  result.classList.add("hidden"); home.classList.remove("hidden");
});
function startQuiz() {
  questions = [...questionSets[category.value]].sort(() => Math.random() - 0.5);
  answers = Array(questions.length).fill(null); current = 0; seconds = 60;
  home.classList.add("hidden"); result.classList.add("hidden"); quiz.classList.remove("hidden");
  document.getElementById("categoryLabel").textContent = category.options[category.selectedIndex].text;
  showQuestion(0); clearInterval(timerId);
  timerId = setInterval(() => {
    seconds--; timerLabel.textContent = seconds + "s";
    if (seconds <= 0) finishQuiz();
  }, 1000);
}
function showQuestion(index) {
  if (index < 0 || index >= questions.length) return;
  current = index; const item = questions[current];
  questionText.textContent = item.q; questionCount.textContent = `Question ${current + 1} of ${questions.length}`;
  progressBar.style.width = `${((current + 1) / questions.length) * 100}%`;
  optionsBox.innerHTML = "";
  item.o.forEach((option, i) => {
    const button = document.createElement("button");
    button.className = "option" + (answers[current] === i ? " selected" : "");
    button.textContent = `${String.fromCharCode(65 + i)}. ${option}`;
    button.addEventListener("click", () => { answers[current] = i; showQuestion(current); });
    optionsBox.appendChild(button);
  });
  document.getElementById("prevBtn").disabled = current === 0;
  document.getElementById("nextBtn").textContent = current === questions.length - 1 ? "Submit Quiz" : "Next";
  timerLabel.textContent = seconds + "s";
}
function finishQuiz() {
  clearInterval(timerId); quiz.classList.add("hidden"); result.classList.remove("hidden");
  const score = questions.reduce((total, item, i) => total + (answers[i] === item.a ? 1 : 0), 0);
  document.getElementById("score").textContent = `${score}/${questions.length}`;
  document.getElementById("feedback").textContent = score >= 4 ? "Excellent work! Keep learning." : "Good effort! Review the topics and try again.
}
```

4.3 Project Execution

• Save index.html, style.css and script.js in the same folder.

• Open index.html in Google Chrome, Microsoft Edge or another modern browser.

• Choose a category from the drop-down list and click Start Quiz.

• Read the question and select one of the four options.

• Use Previous or Next to move between questions. The selected answer remains saved.

• Click Submit Quiz on the final question, or wait for the timer to finish.

• Read the score and feedback. Click Try Again to return to the home page.

Input: Selected category and answers chosen by the user. **Processing:** JavaScript displays questions, tracks answers and time, and compares answers with the answer key. **Output:** Final score and feedback displayed on the result screen.

5. Output Screenshots

The following interface previews illustrate the expected home screen, question screen and result screen. They are report illustrations; after opening the supplied project files in a browser, capture actual screenshots of the running application for final submission if your faculty requires live output evidence.

Figure 1: Home page and category selection

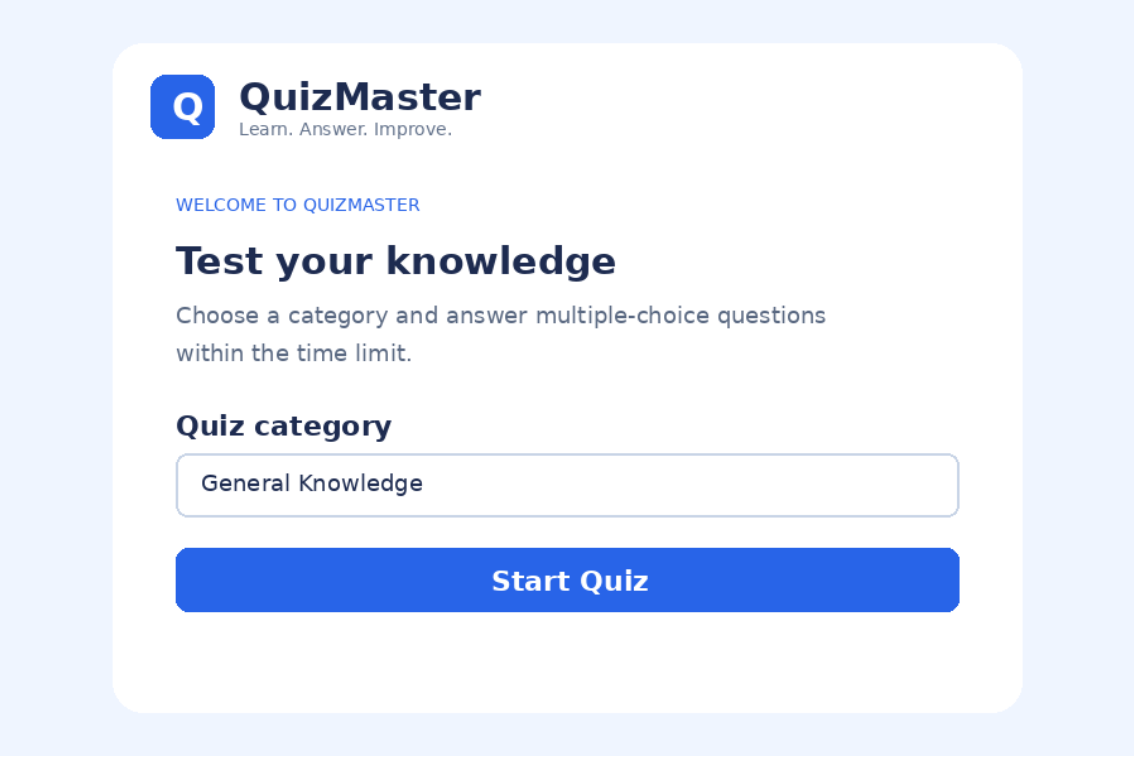


Figure 2: Quiz question, timer and navigation

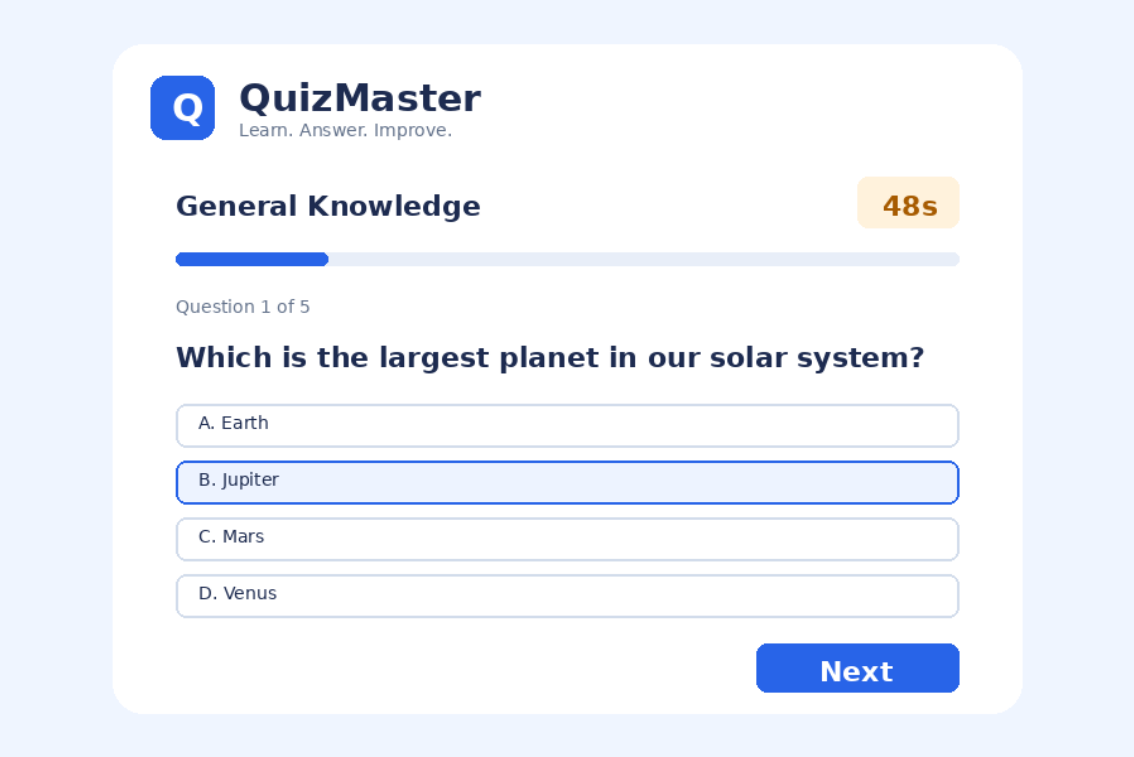


Figure 3: Result screen and restart option



6. Technical Explanation

Document structure: HTML uses semantic sections and controls to separate the home screen, quiz interface and result display. The hidden class switches between these views.

Styling and responsive layout: CSS uses flexbox, grid, spacing, rounded cards and a media query to keep the interface usable on smaller screens.

Data representation: JavaScript stores each question as an object containing a question string, an array of options and the index of the correct answer. The category object groups question arrays by topic.

Event handling: Click event listeners respond to Start, Previous, Next, option selection and Try Again actions. The selected answer is stored in an array matching the current question index.

Timer: setInterval reduces the remaining seconds once each second. When time reaches zero, finishQuiz() is called and the timer is cleared.

Score calculation: The reduce() method counts answers whose selected option index matches the correct answer index. The final score is shown as a fraction out of five.

Limitations: The demonstration uses a small fixed question bank and stores no account or history data. Refreshing the browser restarts the attempt. The timer is client-side and is intended for practice rather than secure examinations.

7. Conclusion

QuizMaster demonstrates the main features of a basic online quiz application. Users can select a category, answer randomized multiple-choice questions, navigate through the quiz, watch the timer and view their score immediately. The project provides practical experience in HTML page structure, CSS design, JavaScript events, arrays, objects, timers and basic application logic.

8. Future Enhancements

• Add more categories and a larger question bank.

• Store scores and quiz history using local storage or a database.

• Add user registration and login with secure authentication.

• Allow administrators or teachers to add and edit questions.

• Show explanations for correct answers and topic-wise performance.

• Add difficulty levels, adjustable time limits and accessibility improvements.

• Deploy the project online and add its real GitHub repository link to the report.

Project Files

The accompanying ZIP contains index.html, style.css and script.js. Keep all three files together in one folder before running the project. The GitHub link is intentionally left as a placeholder because a real repository link can only be added after the files are uploaded to the user's GitHub account.